Desenvolvimento de Sistemas para Dispositivos Móveis

Corrotinas, Flow e ViewModel

DIM0524 — Turma 01 · Sprint 2 · aula de 28/10

Prof. Fernando · UFRN · 2026.2

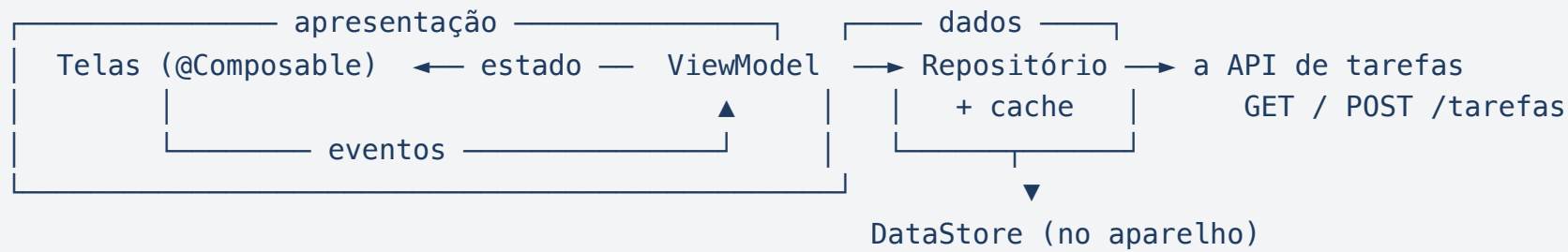
A Sprint 2, a última

	Data
Conteúdo	hoje (passos 11 e 12) e em vídeo, até 30/10 (passos 13 a 16)
Revisão para a prova	04/11 (presencial) e 09/11 (online)
Acompanhamento online	16/11
Prova escrita	**11/11 (quarta)**, em laboratório
Daily meetings online	23 e 25/11 (presencial, se preferirem, em 18/11)
Entrega final	**30/11, 23:59**

> **Prova (11/11):** presencial, no laboratório, no horário da aula, no **Multiprova**, com questões fechadas sobre todo o conteúdo do curso. É permitida **uma folha A4 manuscrita, frente e verso**; nenhum outro material pode ser consultado.

Critério	Peso	Aula
Estado e arquitetura	25%	hoje e vídeo
Estados da interface	15%	hoje
Dados reais e erro de rede	25%	vídeo
Persistência local	10%	vídeo

Onde vamos chegar



- O estado e as decisões saem da tela e vão para um ViewModel, que se testa sem abrir a tela
- As tarefas vêm da API de tarefas, pela rede; sem rede, da última lista guardada
- Cada estado da tela (carregando, vazio, erro, sucesso) é um caso de uma classe selada

Branch `aula`: `./passo.sh 10` é onde a Sprint 1 terminou; hoje vamos até o 12.

Como acompanhar: as tags

Cada conceito tem uma tag no branch `aula`

Conceito	Tag
ViewModel com `StateFlow` (Passo 11)	`passo-11`
Corrotinas e estados selados (Passo 12)	`passo-12`

No Codespace: `./passo.sh 11` ou `./passo.sh 12`; `mise run app` abre o app e `mise run test` roda os testes.

Corrotinas em uma página

Já vistas na aula de Kotlin (`suspend`). O essencial para hoje:

Peça	O que faz
`suspend fun`	uma função que pode esperar (rede, disco) sem travar a thread
`launch { }`	inicia uma corrotina e segue em frente; devolve um `Job`
escopo (`CoroutineScope`)	dono das corrotinas: quando ele termina, as corrotinas dele são canceladas
`viewModelScope`	o escopo do ViewModel: cancelado quando o ViewModel deixa de existir
`Dispatchers.Main`	a thread da interface; as corrotinas do ViewModel começam nela

```
viewModelScope.launch {
    val tarefas = repositorio.listar()   // suspende aqui; a tela continua respondendo
    _estado.update { it.copy(...) }      // volta e atualiza o estado
}
```

> Concorrência estruturada: toda corrotina tem um dono. Fechar a tela cancela o que ela pediu, sem vazamento.

📖 **Ref.** Android — Kotlin coroutines on Android

O que é (e o que não é) uma corrotina

- **Não é** uma thread: milhares de corrotinas cabem numa thread só.
- **É** uma computação que pode **suspender** e **retomar**, sem prender a thread.
- `suspend` marca o ponto em que a função pode ceder a thread; nada roda "em paralelo" só por ser `suspend`.

	`Thread`	`Corrotina`
custo de criar	alto (pilha do SO)	baixo (um objeto)
quem agenda	o sistema operacional	o despachante da linguagem
troca de contexto	o SO	pontos de suspensão, cooperativo

📖 **Ref.** Kotlin — coroutines

Por que a tela não pode esperar

```
thread principal:  [desenha][toque][desenha][ ---- espera a rede, 2 s ---- ][desenha]
                                              a tela congela; no Android, 5 s viram "o app não responde"

com corrotina:     [desenha][toque][desenha][desenha][desenha][desenha][atualiza a lista]
                                    └── a busca suspende e volta quando a resposta chega
```

- A tela é redesenhada a cada ~16 ms (60 quadros por segundo): tudo o que roda na thread principal disputa esse tempo
- **Bloquear** prende a thread até a resposta chegar; **suspender** libera a thread e continua a função depois, do mesmo ponto
- `suspend` não cria thread nem deixa nada mais rápido: só permite esperar sem prender

Concorrência estruturada

```
viewModelScope ─┬─ launch { carregar() }      ← cancelada quando o ViewModel deixa de existir
                └─ launch { adicionar() }
```

Situação	O que acontece
o usuário sai da tela antes da resposta	o ViewModel é descartado, o escopo é cancelado, e a busca para
uma corrotina filha falha	o erro sobe para o escopo, em vez de sumir em silêncio
o escopo é cancelado	cada corrotina recebe uma `CancellationException` no próximo ponto de suspensão

```
try { repositorio.listar() }
catch (e: CancellationException) { throw e }   // deixa o cancelamento seguir
catch (e: Exception) { /* erro de verdade */ }
```

`GlobalScope.launch` não tem dono: ninguém cancela. No app, as corrotinas nascem no `viewModelScope`.

O dono cancela a corrotina

- Toda corrotina tem um **dono** (um escopo): no app, é o `viewModelScope`.
- Quando o dono morre (o usuário fecha a tela), as corrotinas dele são **canceladas**.
- Cancelar não é erro: a `CancellationException` é o sinal de "pare", e deve **propagar**, não ser engolida.

> Um `catch (e: Exception)` genérico que engole a `CancellationException` quebra o cancelamento — e o próximo bloco mostra o jeito certo.

Cancelamento é cooperativo

- Cancelar **não mata** a corrotina: pede para ela parar no próximo ponto de suspensão.
- Uma corrotina que faz trabalho pesado sem `suspend` não vê o cancelamento.
- Por isso toda função `suspend` deve ser "cancelável": consultar ou propagar o cancelamento, nunca engolir.

```
try { repositorio.listar() }
catch (e: CancellationException) { throw e }   // deixa o cancelamento seguir
catch (e: Exception) { /* erro de verdade */ }
```

📖 **Ref.** Kotlin — cancellation and timeouts

Em que thread a corrotina roda

Despachante	Para quê
`Dispatchers.Main`	atualizar a interface; é onde o `viewModelScope` começa
`Dispatchers.Default`	cálculo pesado, que usa CPU (ordenar milhares de itens, processar imagem)
`Dispatchers.IO`	chamadas que **bloqueiam** (arquivo com `java.io`, JDBC), só nos alvos JVM e nativos

```
val ordenadas = withContext(Dispatchers.Default) {   // troca de thread e volta com o resultado
    tarefas.sortedBy { it.titulo }
}
```

- Funções `suspend` bem-feitas são seguras para chamar da Main: o Ktor Client e o DataStore trocam de thread por conta própria
- Por isso o exemplo não tem `withContext`: o ViewModel chama o repositório direto da Main
- No desktop, a Main é a thread do Swing: vem de `kotlinx-coroutines-swing`, acrescentada no Passo 11

Flow e StateFlow

	O que é	No app
`Flow<T>`	uma sequência de valores que chegam ao longo do tempo	a lista atualizada a cada mudança
`StateFlow<T>`	um `Flow` que sempre tem um valor atual, e emite quando ele muda	o estado da tela
`MutableStateFlow<T>`	o `StateFlow` que pode ser alterado	privado, dentro do ViewModel

```
private val _estado = MutableStateFlow(EstadoLista())     // só o ViewModel altera
val estado: StateFlow<EstadoLista> = _estado.asStateFlow()  // a tela só lê

_estado.update { it.copy(texto = novo) }                  // um valor novo, imutável
```

- O estado é uma `data class` imutável: mudar é criar uma cópia com `copy`
- `update { }` faz a troca de forma segura, mesmo com várias corrotinas

📖 **Ref.** Android — StateFlow and SharedFlow

Flow frio, StateFlow quente

```
val contagem: Flow<Int> = flow {
    for (i in 1..3) { delay(1000); emit(i) }    // só roda quando alguém coleta
}

contagem.map { it * 10 }.collect { println(it) }  // 10, 20, 30, um por segundo
```

	Flow (frio)	StateFlow (quente)
quando roda	a cada `collect`, do começo	existe antes e depois dos coletores
valor atual	não tem	sempre tem (`.value`)
quem chega depois	recebe a sequência inteira	recebe o valor atual e as mudanças
valor igual ao anterior	é emitido	não é emitido

- Operadores (`map`, `filter`, `combine`) montam um `Flow` novo sem executar nada: a execução começa no `collect`
- Na Sprint 2, os `Flow` aparecem no DataStore (`data`) e nos testes com Turbine

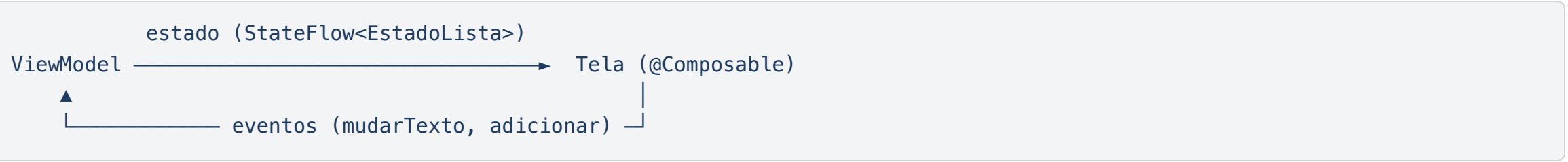
`StateFlow`: um valor + as mudanças

- `StateFlow` é um `Flow` **com estado**: sempre tem um `.value` e emite só quando o valor **muda**.
- É o encaixe certo para o estado da tela: a tela quer o valor **atual** e as **mudanças**.
- `MutableStateFlow` é privado; `asStateFlow()` expõe só leitura.

```
private val _estado = MutableStateFlow(EstadoLista())
val estado = _estado.asStateFlow()   // StateFlow<EstadoLista>: só leitura
```

📖 **Ref.** Android — StateFlow and SharedFlow

Fluxo unidirecional de dados



- O estado desce; os eventos sobem. A tela nunca altera o estado direto
- É a elevação de estado da Sprint 1, um nível acima: quem guarda o estado agora é o ViewModel, e não uma função `@Composable`
- Para saber por que a tela está assim, basta olhar uma fonte: o `StateFlow`
- As telas (`TelaLista`, `TelaDetalhe`) não mudam: continuam recebendo valores e funções

Estado desce, evento sobe

- A tela **lê** o estado (`collectAsStateWithLifecycle`) e **emite eventos** (`viewModel.adicionar()`).
- A tela nunca altera o estado direto.
- Quem decide o que muda é o **ViewModel**, que concentra as regras.

> O `MutableStateFlow` fica **privado**; para fora, só o `StateFlow` (somente leitura). Confiram na tag `passo-11`.

Passo 11

O estado sai da tela: ViewModel com `StateFlow`

Antes: o estado dentro de `Conteudo`

```
@Composable
fun Conteudo(largo: Boolean, ...) {
    var tarefas by remember { mutableStateOf(tarefasIniciais) }
    var texto by rememberSaveable { mutableStateOf("") }
    var proximoId by remember { mutableStateOf(3) }
    var selecionada by rememberSaveable { mutableStateOf<Int?>(null) }

    val adicionar = {
        tarefas = tarefas.comNova(proximoId, texto)
        proximoId++
        texto = ""
    }
    ...
}
```

- Funciona, mas as decisões (validar, gerar o id, limpar o texto) moram numa função de interface
- Testar `adicionar` exige montar a tela e clicar
- Não há onde iniciar uma busca que demora: o Passo 12 precisa de um escopo de corrotinas que não seja a composição

O ViewModel

```
data class EstadoLista(
    val tarefas: List<Tarefa> = tarefasIniciais,
    val texto: String = "",
)

class TarefasViewModel : ViewModel() {
    private val _estado = MutableStateFlow(EstadoLista())
    val estado: StateFlow<EstadoLista> = _estado.asStateFlow()
    private var proximoId = 3

    fun mudarTexto(texto: String) = _estado.update { it.copy(texto = texto) }

    fun adicionar() = _estado.update {
        if (!tituloValido(it.texto)) it
        else it.copy(tarefas = it.tarefas.comNova(proximoId++, it.texto), texto = "")
    }

    fun alternar(id: Int) = _estado.update { it.copy(tarefas = it.tarefas.alternando(id)) }
}
```

As regras (`tituloValido`, `comNova`, `alternando`) são as funções puras da Sprint 1: o ViewModel só as chama.

O que fica no ViewModel e o que fica na tela

No ViewModel (StateFlow)	Na tela (remember / rememberSaveable)
a lista de tarefas	a tarefa selecionada no painel lateral
o texto do formulário	a posição de rolagem, o foco, animações
carregando, erro, vazio, sucesso	o que só muda a aparência
as regras: validar, gerar id, chamar o repositório	—

- A pergunta é: "isso é dado do app ou só da interface?"
- No exemplo, `selecionada` continua em `rememberSaveable` dentro de `Conteudo`: é estado só da interface
- O ViewModel não recebe `NavController`, `Context` nem tipos do Compose: só dados e o repositório

A tela lê o estado e chama eventos

```
@Composable
fun Conteudo(
    largo: Boolean,
    viewModel: TarefasViewModel = viewModel { TarefasViewModel() },   // vem de fora: testável
) {
    val estado by viewModel.estado.collectAsStateWithLifecycle()

    TelaLista(
        estado.tarefas, estado.texto,
        viewModel::mudarTexto, viewModel::adicionar, viewModel::alternar,   // referências a funções
        onAbrir = { ... },
    )
}
```

Antes (Sprint 1)	Agora
`var tarefas by remember { mutableStateOf(...) }` na tela	`StateFlow` no ViewModel
as regras só se testam pela interface	o ViewModel se testa sozinho, sem tela

`collectAsStateWithLifecycle` para de coletar quando a tela sai de vista.

📖 **Ref.** Compose Multiplatform — ViewModel

Testar o ViewModel sem tela

```
// O ViewModel sem tela: cada evento muda o estado, e o teste lê estado.value.
class TarefasViewModelTest {

    @Test
    fun adicionarAcrescentaALimpaOTexto() {
        val vm = TarefasViewModel()
        vm.mudarTexto("Estudar Flow")
        vm.adicionar()
        assertEquals("Estudar Flow", vm.estado.value.tarefas.last().titulo)
        assertEquals("", vm.estado.value.texto)
    }

    @Test
    fun tituloEmBrancoNaoAcrescenta() { ... }

    @Test
    fun alternarMarcaATarefa() { ... }
}
```

O teste monta o ViewModel sem tela e lê `estado.value`, o valor atual do `StateFlow`, sem precisar coletar. Como `mudarTexto`, `adicionar` e `alternar` atualizam o estado na hora, o valor reflete cada evento imediatamente.

```
./passo.sh 11
./gradlew :composeApp:desktopTest     # 11 testes: regras, telas e o ViewModel
```

Erros comuns no Passo 11

Erro	O que acontece	Como fica
`val estado = MutableStateFlow(...)` público	a tela consegue alterar o estado por fora	`_estado` privado e `asStateFlow()`
`TarefasViewModel()` direto no `@Composable`	um ViewModel novo a cada recomposição: o estado some	`viewModel { TarefasViewModel() }`
mudar uma `MutableList` dentro do estado	o objeto é o mesmo, e o `StateFlow` não emite	lista imutável e `copy`
lifecycle numa versão e Navigation em outra	"State must be at least CREATED" ao voltar de uma tela	versões da mesma família (2.9.x)
ler com `.value` dentro do `@Composable`	a tela não recompõe quando o estado muda	`collectAsStateWithLifecycle()`

O ViewModel sobrevive à recomposição

- A tela é recriada a cada rotação; o `ViewModel` **não** — ele fica ligado ao escopo da tela, não à composição.
- Por isso o estado no `ViewModel` **sobrevive** onde o `remember` se perderia.
- No exemplo, `selecionada` (estado só da interface) continua no `rememberSaveable`: cada coisa no seu lugar.

📖 **Ref.** Compose Multiplatform — ViewModel

Passo 12

Carregar sem travar a tela: estado selado

Os quatro estados como classe selada

```
sealed interface EstadoTela {
    data object Carregando : EstadoTela
    data object Vazio : EstadoTela
    data class Sucesso(val tarefas: List<Tarefa>) : EstadoTela
    data class Erro(val mensagem: String) : EstadoTela
}
```

```
when (val tela = estado.tela) {                  // sem else: o compilador confere os quatro
    EstadoTela.Carregando -> TelaCarregando()
    is EstadoTela.Erro -> TelaErro(tela.mensagem, onTentarDeNovo = viewModel::carregar)
    EstadoTela.Vazio, is EstadoTela.Sucesso -> TelaLista(estado.tarefas, ...)
}
```

- Classe selada: todos os casos são conhecidos na compilação. Um quinto estado faz o `when` deixar de compilar até alguém decidir como mostrá-lo
- Dentro do ramo `is EstadoTela.Erro`, `tela` já é um `Erro` (*smart cast*): `tela.mensagem` existe
- É o critério "Estados da interface" da rubrica: carregando, erro, vazio e sucesso

A classe selada força a decisão

- `EstadoTela` é uma classe selada: `Carregando`, `Vazio`, `Sucesso` e `Erro`.
- O `when` sobre ela **não tem** **`else`**: um estado novo faz o código **deixar de compilar** até alguém decidir como mostrá-lo.

> O compilador vira a memória de que "falta tratar aquele estado". Confiram na tag `passo-12`.

O estado da tela, com o conteúdo dentro

```
// O estado da tela de lista: o conteúdo (um dos quatro casos) e o texto do formulário.
data class EstadoLista(
    val tela: EstadoTela = EstadoTela.Carregando,
    val texto: String = "",
) {
    val tarefas: List<Tarefa> get() = (tela as? EstadoTela.Sucesso)?.tarefas.orEmpty()
}
```

- O formulário existe em qualquer caso; a lista só existe no `Sucesso`
- `tarefas` é calculada a partir de `tela`: não há duas fontes para a mesma informação
- `as?` devolve `null` quando `tela` não é `Sucesso`; `orEmpty()` troca o `null` por lista vazia
- O estado começa em `Carregando`: a primeira coisa que a tela mostra é o indicador

Um repositório que demora de propósito

```
interface RepositorioDeTarefas {
    suspend fun listar(): List<Tarefa>        // a resposta pode demorar (rede, disco)
    suspend fun adicionar(titulo: String): Tarefa
}

class RepositorioEmMemoria(
    private val demora: Duration = 800.milliseconds,
    var falhar: Boolean = false,
    iniciais: List<Tarefa> = tarefasIniciais,
) : RepositorioDeTarefas {
    override suspend fun listar(): List<Tarefa> {
        delay(demora)          // suspende a corrotina; a thread da interface continua livre
        if (falhar) error("sem conexão")
        return tarefas.toList()
    }
    ...
}
```

- `delay` suspende; `Thread.sleep` bloquearia a thread da interface
- É a porta que o vídeo vai implementar com a rede (Passo 14) e com cache (Passo 15)

Carregar numa corrotina

```
fun carregar() {
    _estado.update { it.copy(tela = EstadoTela.Carregando) }
    viewModelScope.launch {
        val tela = try {
            val tarefas = repositorio.listar()                  // 800 ms no repositório em memória
            if (tarefas.isEmpty()) EstadoTela.Vazio else EstadoTela.Sucesso(tarefas)
        } catch (e: CancellationException) {
            throw e                                              // cancelamento não é erro
        } catch (e: Exception) {
            EstadoTela.Erro("Não foi possível carregar as tarefas.")
        }
        _estado.update { it.copy(tela = tela) }
    }
}
```

- O repositório em memória tem 800 ms de demora e um `falhar`: é possível ver os quatro estados sem rede
- `CancellationException` é como a corrotina é cancelada: capturá-la e seguir em frente impede o cancelamento

As telas de carregando e de erro

```
@Composable
fun TelaCarregando(modifier: Modifier = Modifier) {
    Box(modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
        CircularProgressIndicator(Modifier.semantics { contentDescription = "Carregando tarefas" })
    }
}

@Composable
fun TelaErro(mensagem: String, onTentarDeNovo: () -> Unit, modifier: Modifier = Modifier) {
    Column(modifier.fillMaxSize().padding(16.dp), verticalArrangement = Arrangement.Center,
           horizontalAlignment = Alignment.CenterHorizontally) {
        Text(mensagem)
        Spacer(Modifier.height(12.dp))
        Button(onClick = onTentarDeNovo) { Text("Tentar de novo") }
    }
}
```

- O indicador tem descrição: o leitor de tela anuncia "Carregando tarefas"
- Toda tela de erro dá uma saída ao usuário; aqui, **Tentar de novo** chama `carregar()`
- O estado vazio é a `TelaLista` sem itens, com "Nenhuma tarefa ainda. Que tal criar a primeira?"

Demonstração

```
./passo.sh 12
mise run app          # ou: ./gradlew :composeApp:hotRunDesktop --auto
```

1. O app abre com o indicador de carregamento e, 800 ms depois, a lista
2. No `RepositorioEmMemoria`, `falhar = true`: aparece a mensagem e o botão **Tentar de novo**
3. Com a lista inicial vazia: aparece "Nenhuma tarefa ainda"

```
./gradlew :composeApp:desktopTest     # agora com os testes de carregando, vazio e erro
```

Testes com corrotinas

```
@OptIn(ExperimentalCoroutinesApi::class)
class TarefasViewModelTest {
    @BeforeTest fun trocarMain() = Dispatchers.setMain(UnconfinedTestDispatcher())
    @AfterTest fun restaurarMain() = Dispatchers.resetMain()

    private fun vm(falhar: Boolean = false, iniciais: List<Tarefa> = tarefasIniciais) =
        TarefasViewModel(RepositorioEmMemoria(Duration.ZERO, falhar, iniciais))

    @Test fun carregaAsTarefas() {
        assertEquals(2, assertIs<EstadoTela.Sucesso>(vm().estado.value.tela).tarefas.size)
    }
    @Test fun listaVaziaViraVazio() {
        assertEquals(EstadoTela.Vazio, vm(iniciais = emptyList()).estado.value.tela)
    }
    @Test fun falhaViraErro() { assertIs<EstadoTela.Erro>(vm(falhar = true).estado.value.tela) }
}
```

- O `viewModelScope` usa `Dispatchers.Main`, que não existe no teste: `setMain` põe um despachante de teste no lugar
- O repositório recebe `Duration.ZERO`: o teste não espera os 800 ms
- O teste só vê o estado final. Para conferir a sequência (carregando → sucesso), o vídeo traz o Turbine

Para conferir o entendimento

1. Por que `_estado` é privado e a tela recebe um `StateFlow`?
2. O que acontece se um `catch (e: Exception)` engolir a `CancellationException`?
3. Por que o `when` sobre `EstadoTela` não tem `else`?
4. Onde fica a tarefa selecionada no painel lateral, e por quê?
5. Num teste, por que o ViewModel recebe um `RepositorioEmMemoria(Duration.ZERO, ...)`?

No vídeo (até 30/10)

Passo	Assunto
13	Camadas (domínio, dados, apresentação) e injeção de dependência com Koin
14	Dados reais: Ktor Client, DTO e erros de rede com nome, chamando a API de tarefas
15	Abrir sem rede: a última lista guardada com DataStore
16	Testes de ViewModel com Turbine e dublês

Até lá, no projeto do grupo:

- levar o estado de uma tela para um ViewModel com `StateFlow`
- modelar os estados dessa tela como classe selada, com os quatro casos
- escolher a API real que o app vai consumir

Onde estudar depois

- Leitura da sprint: `leituras/moveis-s2.md`, capítulos 1 a 4
- Passos: `exemplos/tarefas-compose/PASSOS.md` · branch `aula`, tags `passo-11` e `passo-12`
- Android — Kotlin coroutines on Android
- Android — StateFlow and SharedFlow
- Compose Multiplatform — ViewModel
- Kotlin — Sealed classes · Coroutines basics